Supplementary material for

"The Model Knows the Textbook but Not Your Class: Curriculum Knowledge Graph Constraints for Slide Generation in Two Vocational-Education Courses"

Suxing Liu

This document holds five evidence blocks that the main text summarises: the ablation results (Section S1), the mutation test of the graph's own structural checks (Section S2), the behavioural probe of the generated code (Section S3), the replication of the claim-bearing conditions on a second vendor's model (Section S4), and the concept-level paired analysis behind the per-baseline tests in §5.4 of the main text (Section S5). All numbers are regenerated by the scripts named in each section from the archived raw outputs; nothing here is hand-transcribed.

S1 Ablation of the contract

Each ablation arm retains the complete contract and rule wording of `full` and deletes exactly one data block, together with the rule text that would leak the deleted information back in: `no_order` removes the ordering block and rule 3, `no_misconception` removes the misconception block and rule 5, and `no_forbidden` removes the forbidden-term block and rule 6. The rule text must be removed together with the data—deleting only the data while keeping a rule that says "you must follow the given order" forces the model to guess an order, and the ablation is no longer clean; a unit test guards this. All four rows come from the ablation batch in `outputs/bench_ablation/`—three sessions × resamplings 1 and 2, $n = 6$ per arm—whose own six `full` re-runs anchor the table. This batch is *separate* from the main experiment, so the `full` row is not directly comparable with the main results table row by row (its coverage reads 1.000 here rather than 0.981, because the single 0.833 run is resampling 3 of U10 in the main batch); only within-table comparisons are made.

One arm in this table is a *null manipulation* and is kept only for completeness. Deleting the ordering block leaves the required-concept list, which is itself emitted in topological order—the same sequence the deleted block restates—so `no_order` still receives the order. Its 0.000 violation rate is therefore not evidence that an ordering constraint is unnecessary; it says only that a second statement of the order can be removed without effect. A clean test of the constraint would have to shuffle the concept list, which we did not do.

Table S1: Ablations; each removes exactly one part of the graph. Mean (standard deviation) over six runs, all from the ablation batch.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑
`full`	1.000 (0.000)	0.000 (0.000)	0.032 (0.078)	1.000 (0.000)
− ordering	1.000 (0.000)	0.000 (0.000)	0.000 (0.000)	1.000 (0.000)
− misconceptions	0.944 (0.086)	0.000 (0.000)	0.040 (0.064)	**0.167** (0.408)
− forbidden terms	1.000 (0.000)	**0.083** (0.129)	0.000 (0.000)	1.000 (0.000)

Regenerated by `python paper/tables.py` from `outputs/bench_ablation/`.

S2 Mutation test of the graph's structural checks

The shipped graph returns zero findings on all ten checks, which on its own is weak evidence: a checker that never fires cannot be distinguished from one that cannot fire. Each of the twelve codes the validator can emit is therefore exercised by one minimal defect injected into a copy of the graph, and Table S2 records whether the owning code responds and what responds with it. All twelve codes fire on their own mutant. A single mutant triggers a second code—an ungrounded skill also violates check 3—which is the correct verdict rather than interference, so the checks are specific as well as sensitive. Regenerated by `python paper/graph_checks.py`.

Table S2: Each of the twelve codes is exercised by one minimal defect injected into a copy of the shipped graph. "Also fires" lists codes that respond in addition to the intended one; a check that failed to fire here would be decorative.

Chk	Code	Injected defect	Also fires
1	`DANGLING_EDGE`	add relates_to edge to undefined node C9999	–
2	`PREREQ_CYCLE`	reverse prerequisites_for C1001→C1003	–
3	`NOT_GROUNDED`	add Concept with no appears_in edge	–
4	`MISSING_EVIDENCE`	strip evidence from is_a C1101→C1001	–
5	`UNIT_CAPACITY`	add 6 filler concepts to U12 (5 present, cap 8)	–
5	`NO_CORE_CONCEPT`	downgrade the 3 mastery concepts of U12	–
6	`DUPLICATE_NAME`	rename C1204 to the name already used by C1001	–
7	`FORBIDDEN_REQUIRED_OVERLAP`	forbid 'circular queue' in U11, where it is a core concept	–
8	`FORBIDDEN_SIBLING_OVERLAP`	forbid 'false overflow' in U11, where it is a sibling recap term	–
9	`ORPHAN_MISCONCEPTION`	add Misconception with no addresses_misconception in-edge	–
10	`IMPLEMENTS_TARGET_TYPE`	point implements at a Misconception node	–
10	`IMPLEMENTS_SOURCE_UNGROUNDED`	implements from a Skill with no appears_in edge	`NOT_GROUNDED`

Five unit tests in `tests/test_graph.py` pin the four self-consistency checks (7–10) to mutants of the shipped graph, so a regression in any of them fails the suite.

S3 Behavioural probe of the generated code

The probe (`paper/behavior_check.py`) extracts every code block from the 54 main runs, strips top-level demo statements with an AST filter—otherwise a demonstration loop leaves `front=2, rear=5` in the namespace and every later probe fails for the wrong reason—executes the remainder in a fresh namespace, and interrogates up to four behaviours: first-in-first-out order, the empty-dequeue signal, the full signal, and survival of repeated enqueue. A block qualifies only if it defines a module, a class, or a function whose first parameter is a queue; fragments such as a bare `enqueue(self, x)` method body are not testable. Class probes are scored separately for bounded and unbounded implementations: an unbounded queue passes the full probe if 64 enqueues complete without error, a bounded one if it eventually raises after at least one successful enqueue. A probe counts as passed only when the code returns a boundary signal (a sentinel, `False`, a raised exception, or a printed message); a silent wrong answer fails.

The single failure is a genuine scaffolding defect rather than a probe artefact: in the U10 `full` run 3 deck, a program-style block writes `enqueue(q, x)` against a capacity constant `CAP`

Table S3: Behavioural probe over all 117 code blocks from the 54 main runs. "Testable" is the share of blocks the harness can execute at all; the rate is the share of testable blocks that satisfy every behaviour asked of them.

Condition	Blocks	Testable	Passed	Rate
`full`	28	9	8	0.89
`rag_mc`	18	6	6	1.00
`rag`	19	6	6	1.00
`placebo`	18	3	3	1.00
`cot`	18	7	7	1.00
`zero_shot`	16	6	6	1.00
All	117	37	36	0.97

that its own slide never defines. This is precisely the "slides are deliberately partial" argument of the main text: the excerpt assumes a surrounding program that a slide does not carry.

As the main text argues, this is a bound and not a metric. The harness is authored by the evaluators, so the evaluator supplies part of the answer it then checks; and the qualifying denominator (0.32 of blocks, unevenly spread) is too small and too condition-dependent to rank arms. `full` passing 8 of 9 against 28 of 28 for the baselines reads as a regression but is not evidence of one.

S4 Cross-model replication

The same generator, prompts, sessions and validator, run on a second vendor's model. This block exists to answer one question—does the contrast that carries the paper (`full` against `rag_mc`) keep its sign off the model it was developed on—and it is not a comparison of model quality.

Two settings differ and are recorded per run in `outputs/bench_kimi/results.csv`: the backbone (`kimi-k3`) and the sampling temperature. Kimi K3 rejects any temperature other than 1.0, so the replication runs at 1.0 against 0.3 for the main table; the value is pinned in `ProviderPreset.fixed_temperature` rather than left to the caller's default, and a unit test holds it there. Within-model contrasts are therefore unaffected, and we compare deltas across models, never levels.

Runs: three sessions × two resamplings, six per condition, except `zero_shot` on Kimi ($n = 5$): one cell could not be generated because the vendor account reached its credit limit, and it is reported as missing rather than refilled. `rag` and `cot` were not re-run on the second backbone; they carry none of the claim and each extra condition costs about seven minutes at the account's concurrency limit of one.

Regenerated by `python paper/tables.py -cross outputs/bench_kimi`.

What replicates. On both metrics that carry the argument, and in the same direction: misconception coverage 1.000 → 0.667 on Kimi against 1.000 → 0.556 on DeepSeek, coverage 1.000 → 0.734 against 1.000 → 0.698. The variance behaviour replicates more exactly than the means do—`full` is 1.000 with zero standard deviation in every run under both models, and `rag_mc`'s standard deviation is 0.516 against 0.527.

What does not. The gain shrinks on the second model (−0.333 against −0.444 for misconceptions, −0.266 against −0.283 for coverage), and on U11 every condition scores 1.000 on Kimi, zero-shot included, so one of the three sessions contributes no contrast at all. The pass-rate gap (67% against 17%) rests on six runs per arm and carries no usable interval. Two backbones are

Table S4: The same conditions under two backbones. Mean (standard deviation) over runs; `rag` and `cot` were not re-run on the second backbone. DeepSeek V4 at temperature 0.3, Kimi K3 at 1.0 (vendor-pinned), so levels are not comparable across the two blocks—only deltas within a block are.

Condition	Coverage ↑	Out-of-graph ↓	Order viol. ↓	Misconception ↑	Pass
DeepSeek V4					
`full`	0.981 (0.056)	0.000 (0.000)	0.020 (0.045)	1.000 (0.000)	67%
`rag_mc`	0.698 (0.295)	0.111 (0.220)	0.007 (0.022)	0.556 (0.527)	22%
`rag`	0.659 (0.248)	0.111 (0.220)	0.030 (0.089)	0.222 (0.441)	0%
`placebo`	0.675 (0.266)	0.167 (0.250)	0.096 (0.131)	0.333 (0.500)	0%
`cot`	0.614 (0.185)	0.019 (0.056)	0.089 (0.183)	0.389 (0.486)	0%
`zero_shot`	0.643 (0.248)	0.019 (0.056)	0.037 (0.089)	0.222 (0.441)	0%
Kimi K3					
`full`	1.000 (0.000)	0.000 (0.000)	0.016 (0.025)	1.000 (0.000)	67%
`rag_mc`	0.734 (0.229)	0.056 (0.136)	0.033 (0.082)	0.667 (0.516)	17%
`placebo`	0.659 (0.188)	0.056 (0.136)	0.028 (0.068)	0.333 (0.516)	0%
`zero_shot`	0.543 (0.270)	0.000 (0.000)	0.000 (0.000)	0.400 (0.548)	0%

two draws from a large space: this rules out an artefact of one vendor's tuning and supports no statement about model families.

S5 Concept-level paired analysis

The main text reports coverage per deck, which leaves three clusters and a clustered interval that includes zero. The contract, however, is written per concept, and the same decks can be scored that way: every required concept is either named in a deck or not, so each (session, resampling, concept) triple is a paired binary observation shared by all conditions. Sixteen required concepts across three sessions and three resamplings give $N = 48$ pairs per baseline.

The matching is the shipped one (validator surface forms and graph aliases, body pages only), so this section adds no new metric—only a different unit of analysis. The test is the exact McNemar test on the discordant cells; the per-unit counts are printed in the main text because the pairs are *not* independent (sixteen concepts nested in three sessions, and the three resamplings of a cell repeat the same concept list), so the p values describe the consistency of the direction, not a population parameter.

Table S5: Concept-level pairs against each baseline, $N = 48$ per row. "both" = covered by both conditions, "`full` only" / "base only" = the discordant cells, RD = paired risk difference. Regenerated by `python paper/concept_pairs.py`.

Baseline	N	both	`full` only	base only	RD	p
`rag_mc`	48	36	11	1	+0.208	6.3×10^{-3}
`rag`	48	34	13	1	+0.250	1.8×10^{-3}
`placebo`	48	35	12	1	+0.229	3.4×10^{-3}
`cot`	48	30	17	1	+0.333	1.4×10^{-4}
`zero_shot`	48	33	14	1	+0.271	9.8×10^{-4}

Reading the pairs. Two features matter. First, every baseline puts more discordant cells on the side of `full` than on its own—11 to 1 against `rag_mc`, 17 to 1 against `cot`—and no baseline reverses the sign, so the direction of the coverage effect does not depend on which control is chosen. Second, no single concept carries it: dropping any one concept from the metric moves a

Table S6: Leave-one-concept-out: the deck-level coverage gap against `rag_mc` is recomputed with each required concept removed in turn, per session. The range is the answer to "is one easy concept carrying the result".

Session	gap, all concepts	range over drops	concepts
U10	-0.056	$[-0.067, 0.000]$	6
U11	$+0.238$	$[+0.111, +0.278]$	7
U12	$+0.667$	$[+0.500, +1.000]$	3

session's gap by at most 0.056, including U12, where three concepts alone produce a gap between $+0.500$ and $+1.000$. What the pairs do *not* fix is U10: its discordant cells are 0 to 1 against `rag_mc` and 3 to 1 against `zero_shot`, so the session-level sign can still flip there, exactly as the clustered interval in the main text says.